

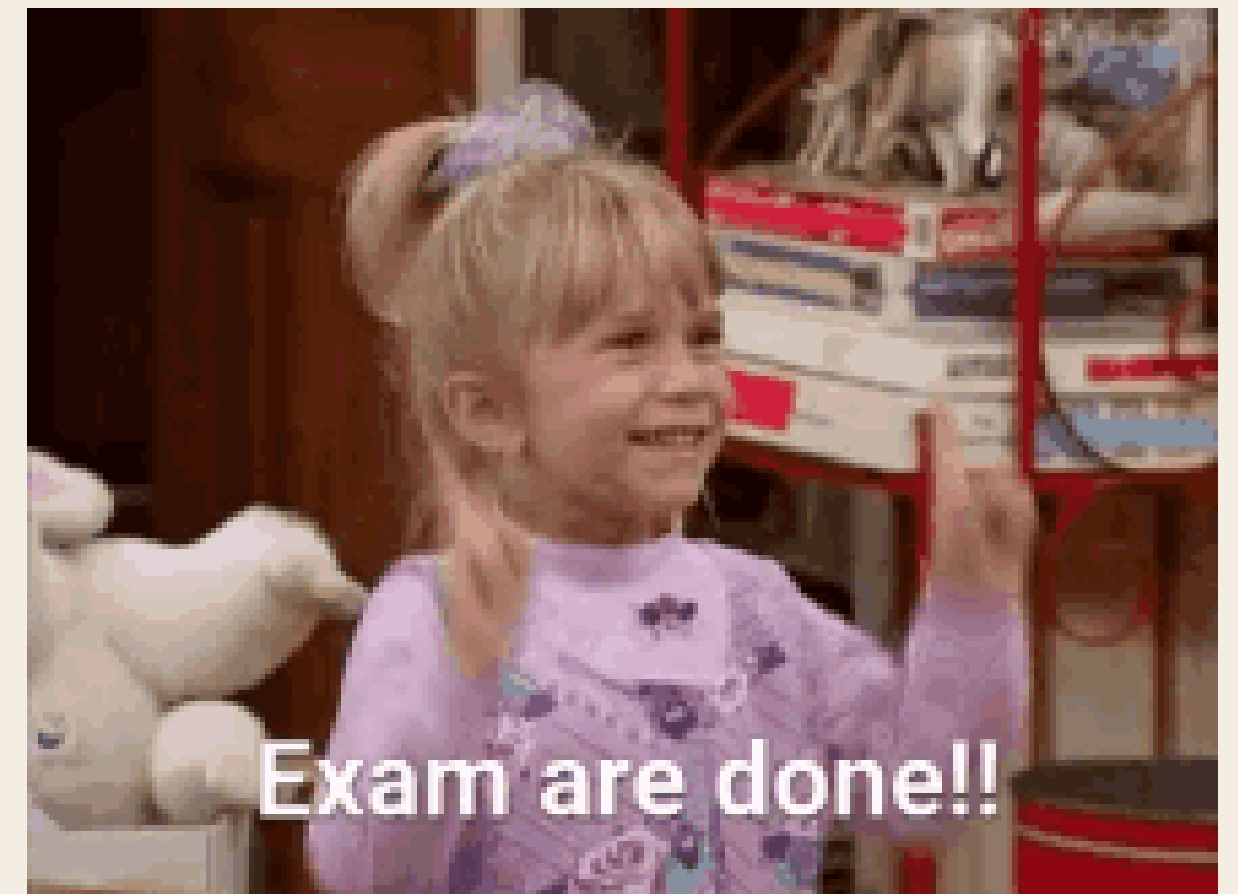
DSA1101

Introduction to Data Science

Week 8 Tutorial 6

TODAY'S AGENDA

1. Quick Revision
2. On-site Questions Attempt
3. On-site Questions Discussion
4. Off-site Questions Discussion



QUICK REVISION

rpart() Parameters

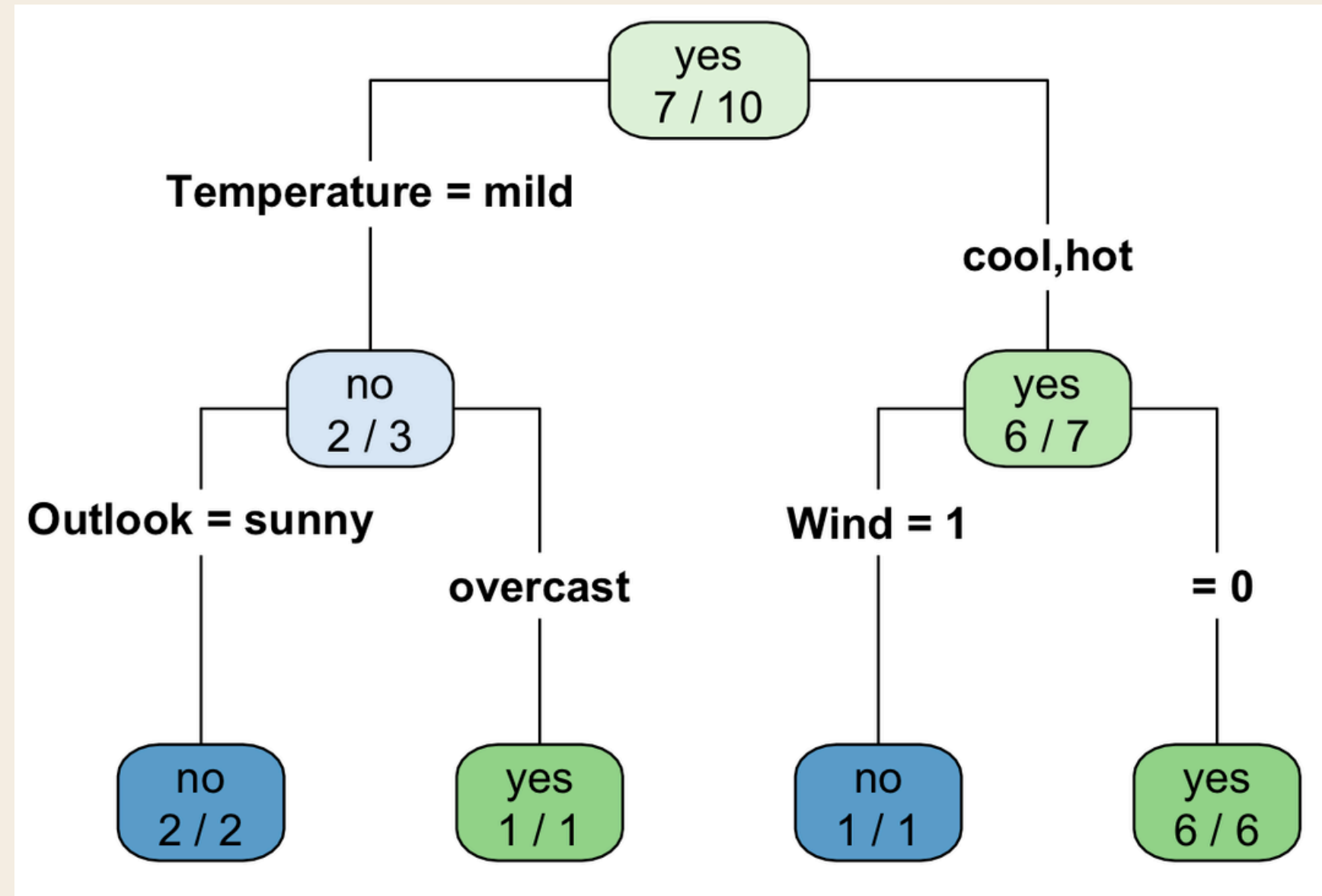
formula (required)	Specifies the response and predictor variables. Format: response ~ predictors <code>Play ~ Outlook + Temperature + Humidity + Wind</code>
data (required)	Data frame containing the variables in <i>formula</i> .
method (optional)	Specifies the type of tree. "class" → classification "anova" → regression "poisson" → count data "exp" → survival data
parms (optional)	Controls splitting behavior (mainly for classification). Format: parms = list(split = "information") Common: split = "gini" (default in R) split = "information" (entropy)
control (optional)	Controls tree growth. Format: control = rpart.control(...) Common: • cp → minimum improvement required to make a split • minsplit → minimum observations needed before splitting • minbucket → minimum observations allowed in a leaf • maxdepth → maximum depth of the tree

rpart.plot() Parameters

x (required)	The fitted rpart model to be plotted.
type	Controls how the tree layout and labels are displayed. 0 → splits shown above nodes 1 → splits inside nodes 2 → label all nodes 4 → clean classification-style layout (very common)
extra	Controls extra information displayed inside nodes. 0 → no extra info 1 → number of observations 2 → class probabilities 104 → predicted class + probability (very common)
varlen	Controls how many characters of variable names are shown in split labels. varlen = 0 → show full variable name
faclen	Controls how many characters of factor levels are shown in labels. faclen = 0 → show full level names
clip.right.labs	Whether long right-side labels are clipped (cut off) to avoid overlapping. TRUE → clip long labels FALSE → show full labels

Example of using `rpart()` and `rpart.plot()` for classification

```
fit <- rpart(Play ~ Outlook + Temperature + Humidity + Wind,  
  method = "class",  
  data = data,  
  control = rpart.control(minsplit=1),  
  parms = list(split='information'))  
  
rpart.plot(fit, type = 4, extra = 2)
```



How to evaluate classification decision tree?

our fav confusion matrix!

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Sometimes Negative and Positive will switch places. So please read carefully before answering!

Accuracy =

$$(TP + TN) / (TP + TN + FP + FN)$$

$$TPR (Recall) = TP / (TP + FN)$$

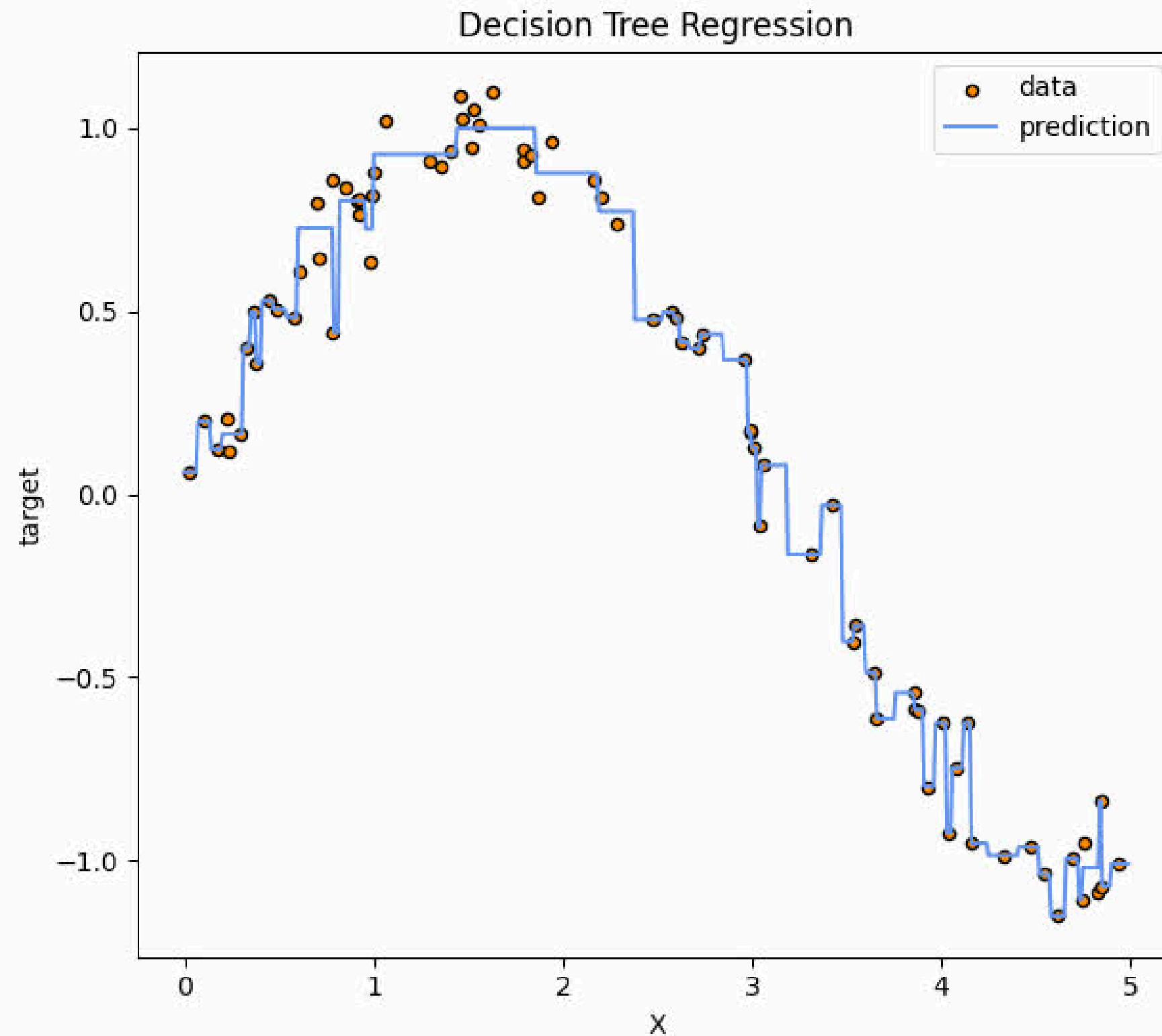
$$FPR = FP / (FP + TN)$$

$$FNR = FN / (TP + FN)$$

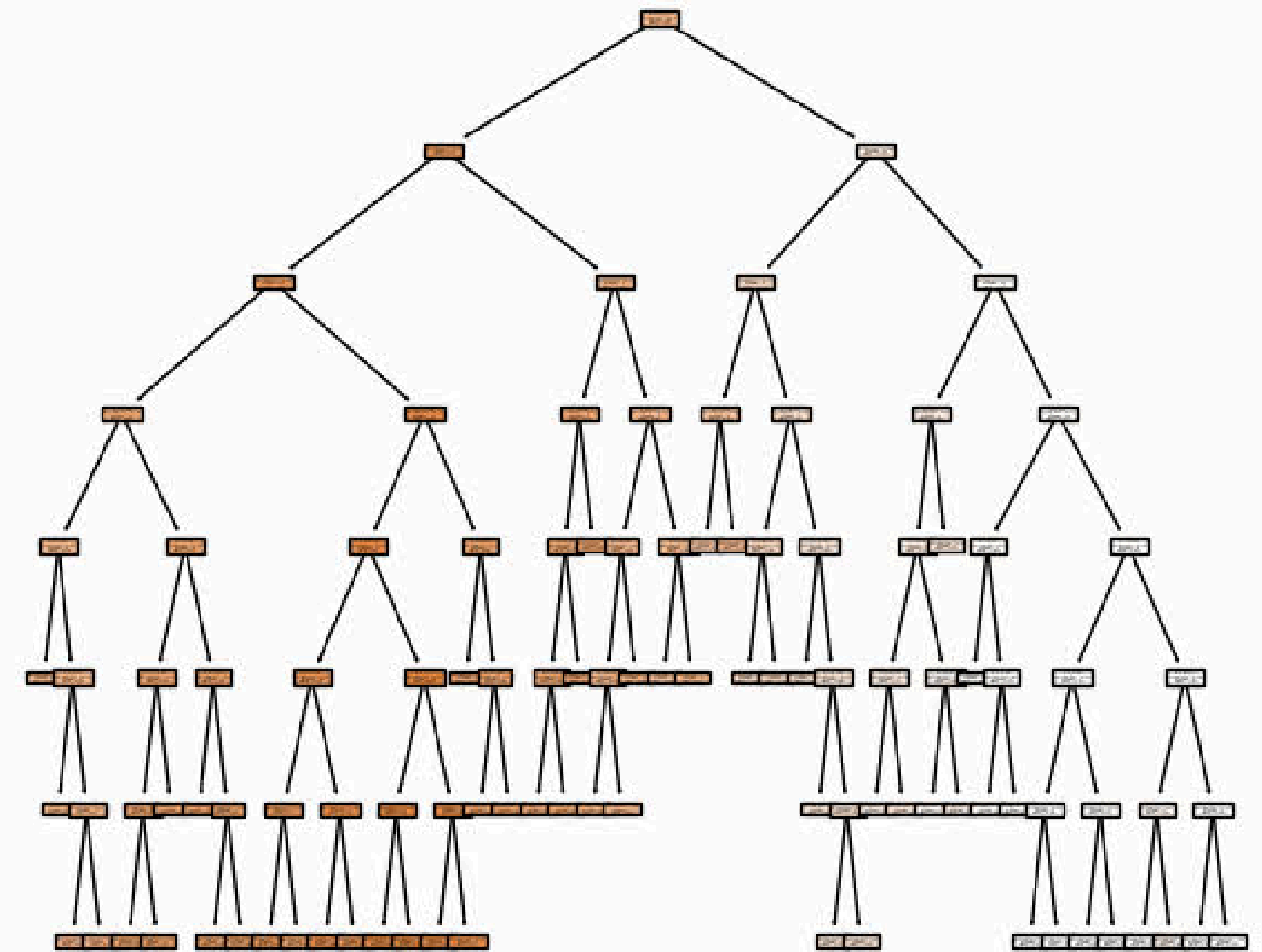
$$Precision = TP / (TP + FP)$$

(Extra) Decision Tree Regressor

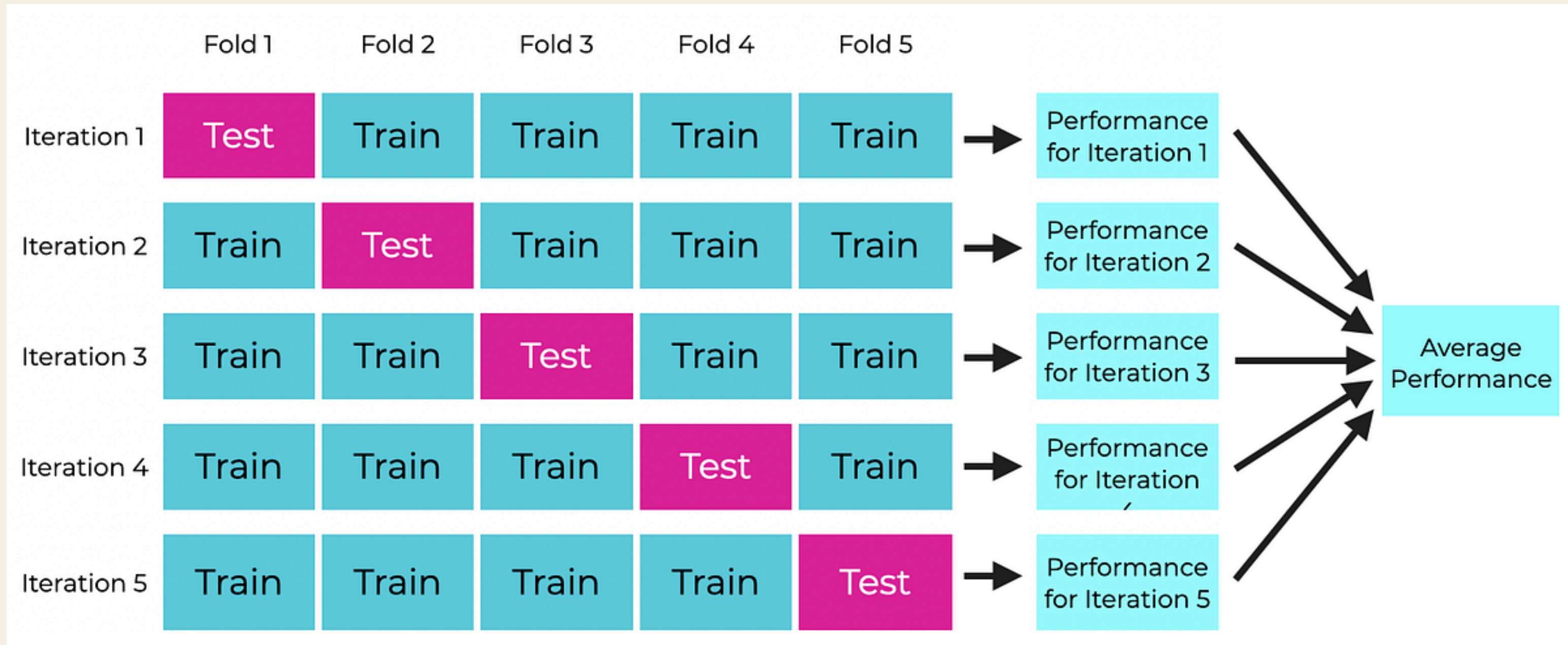
Decision tree regressor with max depth=7



Decision Tree Structure



N-fold Cross Validation



(Extra) Decision Tree Pros vs Cons

Pros	Cons
Easy to interpret (“White-box model”) Eg. If Age < 34.5 → predict churn Else → predict no churn	Overfits easily (start memorizing noise instead of learning patterns) Eg. Age < 34.5; Age < 33.8; Age < 33.2
Works with both numerical and categorical data	High variance (Unstable) Small changes in the data can produce very different trees → (Random Forest, Gradient Boosting)
Little data preparation required (Eg. no need to standardize) Good for quick modelling	Greedy Algorithm They choose the best split at that moment, not the best overall tree.

ON-SITE QUESTIONS

ON-SITE QUESTION

Customer churn is the **loss of clients or customers**. Banks, telephone service, companies, Internet service providers, pay TV companies and insurance firms often use customer churn analysis and customer churn rates as one of their key business metrics.

This is because the cost of retaining an existing customer is far less than acquiring a new one. Companies from these sectors often have customer service branches which attempt to win back defecting clients, because recovered long-term customers can be worth much more to a company than newly recruited clients.

In this problem, a wireless telecommunications company wants to predict whether a customer will churn (switch to a different company) in the next six months. With a reasonably accurate prediction of a person's churning, the sales and marketing groups can attempt to retain the customer by offering various incentives. Variables of our concern are listed below.

- (i) Age (years)
- (ii) Married (true/false)
- (iii) Duration as a customer (years)
- (iv) Churned contacts -Number of the customer's contacts that have churned (count)
- (v) Churned (true/false)—Whether the customer churned

- (a) Build a decision tree for predicting customer churn, using the feature variables Age, Married, Cust_years and Churned_contacts.

Discuss with your group! Time limit: 5 mins

ON-SITE PREP AND INSPECT

```
# Import data  
readLines("Data/churn.csv", n = 3)  
churn = read.csv("Data/churn.csv")
```

```
> head(churn)
```

	ID	Churned	Age	Married	Cust_years	Churned_contacts
1	1	0	61	1	3	1
2	2	0	50	1	3	2
3	3	0	47	1	2	0
4	4	0	50	1	3	3
5	5	0	29	1	1	3
6	6	0	43	1	4	3

Spotted unwanted columns (columns that provide no info)

ON-SITE PREP AND INSPECT

```
# Remove ID column  
churn = churn[,-1]  
head(churn)
```

!! -1 means removing column
1 from data frame

```
> head(churn)
```

	Churned	Age	Married	Cust_years	Churned_contacts
1	0	61	1	3	1
2	0	50	1	3	2
3	0	47	1	2	0
4	0	50	1	3	3
5	0	29	1	1	3
6	0	43	1	4	3

These two columns should be categorical

ON-SITE PREP AND INSPECT

```
# Switch categorical variables to factor
churn$Married = as.factor(churn$Married)
churn$Churned = as.factor(churn$Churned)
```

```
> str(churn)
'data.frame':   8000 obs. of  5 variables:
 $ Churned      : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...
 $ Age          : int   61 50 47 50 29 43 50 29 32 48 ...
 $ Married      : Factor w/ 2 levels "0","1": 2 2 2 2 2 2 1 1 2 2 ...
 $ Cust_years   : int    3 3 2 3 1 4 2 3 3 4 ...
 $ Churned_contacts: int   1 2 0 3 3 3 1 2 3 0 ...
```

ON-SITE QUESTION 1A

Q: Build a decision tree for predicting customer churn, using all features

```
library(rpart)
library(rpart.plot)

# Here we choose minsplit and information gain
fit <- rpart(formula = Churned ~ Age + Married + Cust_years + Churned_contacts,
             data = churn,
             method = "class",
             control = rpart.control(minsplit = 1),
             parms = list(split = 'information'))

# Visualizing Decision Tree
rpart.plot(x = fit, type = 4, extra = 2,
           clip.right.labs = FALSE, varlen = 0, faclen = 0)
```

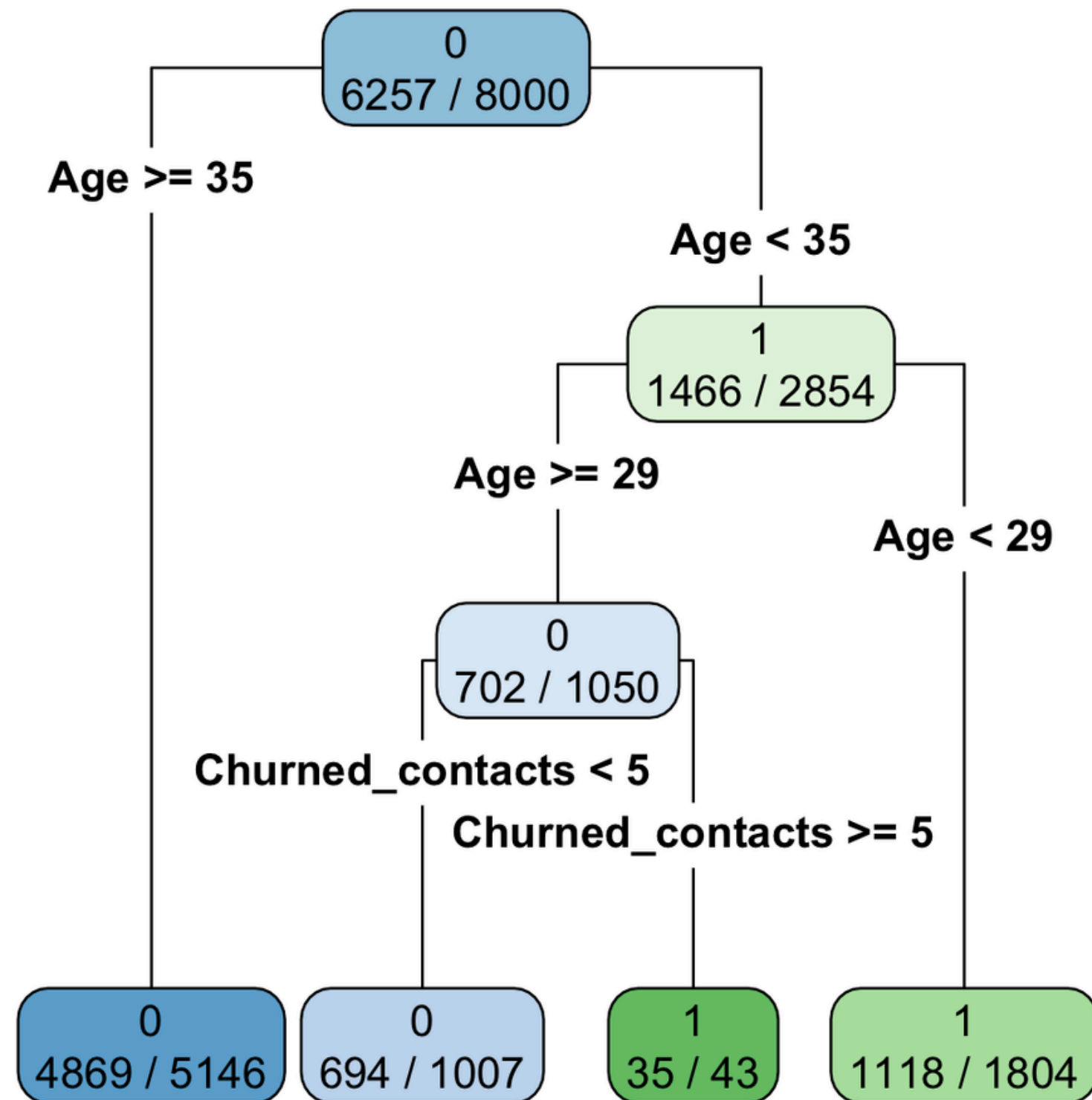

ON-SITE QUESTION 1A

Q: What is the most important feature based on this tree?

TLDR:

Young people are more likely to churn (Age < 29).

Middle-aged customers (Age 29-34) are likely to churn if 5 or more of their contacts have churned.



ON-SITE QUESTION 1B

(b) Consider the decision tree in part (a) to predict binary variable Churned. Use the tree to predict customer churn for the following observations.

	Age	Married	Cust_years	Churned_contacts
2821	26	1	2	2
96	23	1	3	3
5085	56	1	5	2
758	36	1	5	2
487	45	0	2	1
987	28	0	2	2
6061	22	1	3	0
3745	22	0	3	2
4709	60	1	2	1
2769	32	0	3	1

Discuss with your group! Time limit: 5 mins

ON-SITE QUESTION 1B

IF: Oh no! I forgot how to make predictions using R 😭.

Let's try to classify new customers ourselves (without using code)

	Age	Married	Cust_years	Churned_contacts
2821	26	1	2	2
96	23	1	3	3
5085	56	1	5	2
758	36	1	5	2
487	45	0	2	1
987	28	0	2	2
6061	22	1	3	0
3745	22	0	3	2
4709	60	1	2	1
2769	32	0	3	1

TLDR:

Young people are more likely to churn (Age < 29).

Middle-aged customers (Age 29-34) are likely to churn if 5 or more of their contacts have churned.

Prediction: [1,1,0,0,0,1,1,1,0,0]

ON-SITE QUESTION 1B

Prediction: [1,1,0,0,0,1,1,1,0,0]

```
> pred
```

1	2	3	4	5	6	7	8	9	10
1	1	0	0	0	1	1	1	0	0

Matches! This demonstrates why Decision tree is very interpretable.

Even humans can use it to make predictions!

ON-SITE QUESTION 1B

Doing the code way:

```
age = c(26, 23, 56, 36, 45, 28, 22, 22, 60, 32)
married = c(1, 1, 1, 1, 0, 0, 1, 0, 1, 0)
cust = c(2, 3, 5, 5, 2, 2, 3, 3, 2, 3)
contact = c(2, 3, 2, 2, 1, 2, 0, 2, 1, 1)

married = as.factor(married)
```

Step 1: Create vectors of each variable

!! remember to convert married to factor

ON-SITE QUESTION 1B

```
# Create a data frame for the 10 test points.  
# This data frame MUST have the header the same as the name of the factors in the tree.  
names(churn)
```

```
> names(churn)  
[1] "Churned"      "Age"          "Married"      "Cust_years"   "Churned_contacts"
```

```
new = data.frame(Age = age,  
                 Married = married,  
                 Cust_years = cust,  
                 Churned_contacts = contact)  
new
```

Step 2: Create a data frame for the 10 test points

ON-SITE QUESTION 1B

```
> new
```

	Age	Married	Cust_years	Churned_contacts
1	26	1	2	2
2	23	1	3	3
3	56	1	5	2
4	36	1	5	2
5	45	0	2	1
6	28	0	2	2
7	22	1	3	0
8	22	0	3	2
9	60	1	2	1
10	32	0	3	1

Step 2: Create a data frame for the 10 test points

ON-SITE QUESTION 1B

```
pred = predict(fit, newdata = new, type = 'class')  
pred
```

```
> pred  
 1  2  3  4  5  6  7  8  9 10  
1  1  0  0  0  1  1  1  0  0  
Levels: 0 1
```

Step 3: Predict the 10 new test points

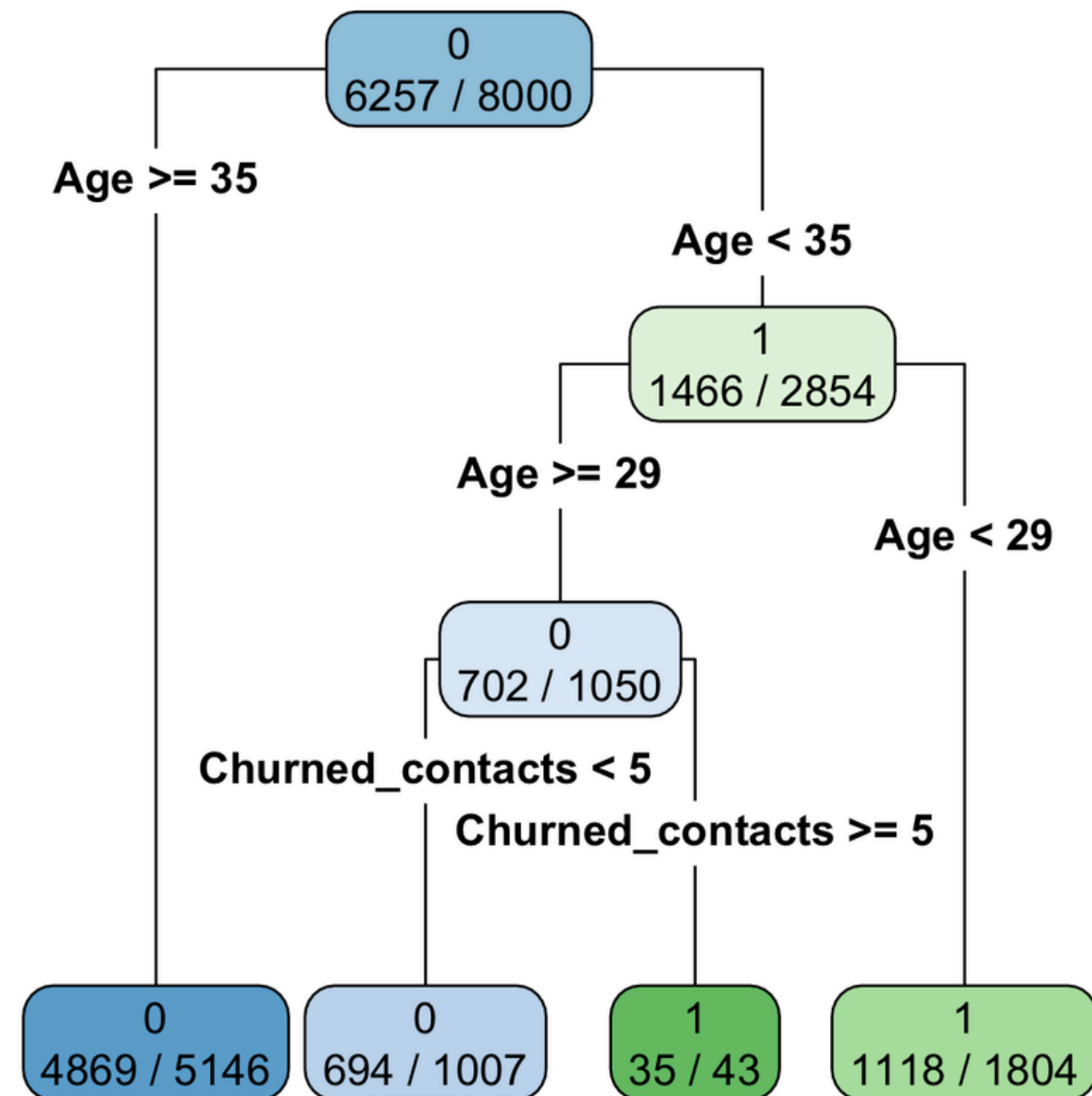
ON-SITE QUESTION 1B

If you're interested to view probability:

```
pred = predict(fit, newdata = new, type = 'prob') # to view probability
pred
```

```
> pred
```

	0	1
1	0.3802661	0.61973392
2	0.3802661	0.61973392
3	0.9461718	0.05382822
4	0.9461718	0.05382822
5	0.9461718	0.05382822
6	0.3802661	0.61973392
7	0.3802661	0.61973392
8	0.3802661	0.61973392
9	0.9461718	0.05382822
10	0.6891758	0.31082423



OFF-SITE QUESTIONS

Let's go to code!

Extra Materials

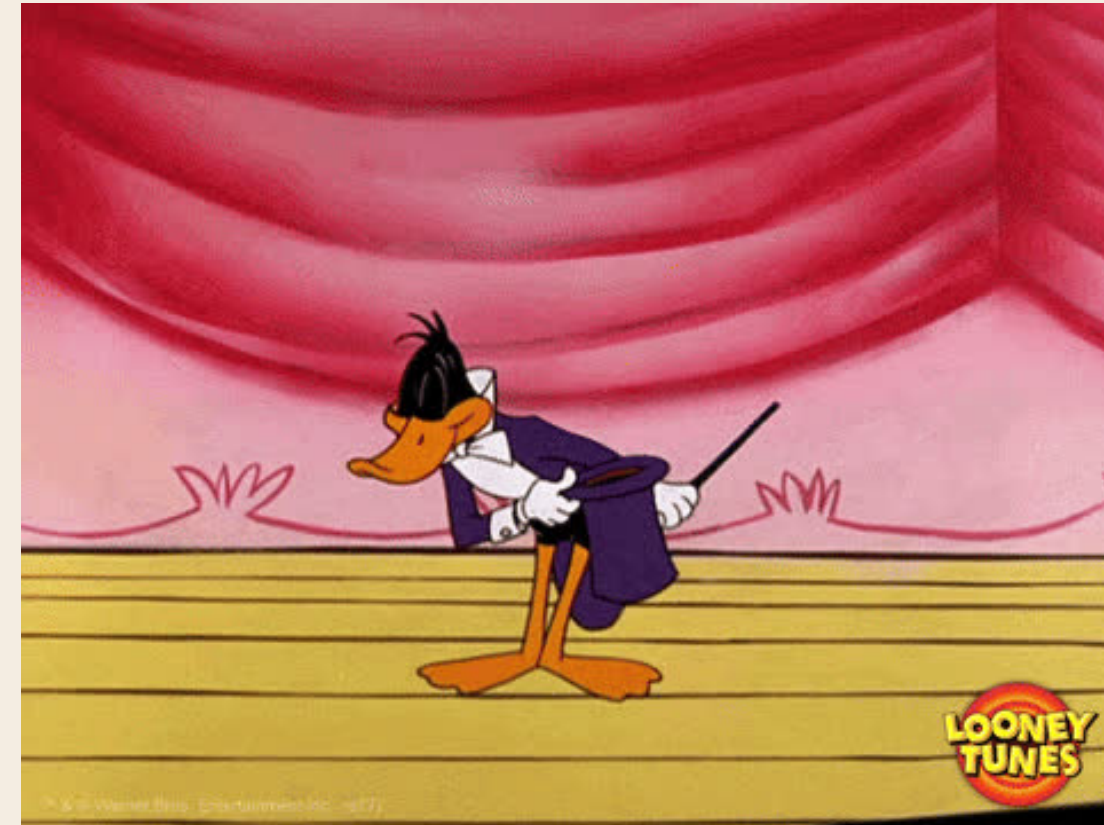
Outside this course, is Decision Tree often a good enough model? **No.**

Prone to overfitting and have high variance

Extra readings:

- **Bagging:** <https://medium.com/@damaniayesh/the-art-of-bagging-enhancing-decision-trees-with-ensemble-techniques-2786e4bffc27>
- **Bagging vs Boosting:** <https://medium.com/data-science-collective/bagging-vs-boosting-a-deep-dive-into-ensemble-learning-for-tree-based-models-b3a061da3e62>

THANK YOU!



Weekly Slides & Code will be shared on Github (★ star it for bookmark):

<https://github.com/kr7001/DSA1101-teaching-materials>